



SELECT scripts explained

The SELECT statement in SQL is a query command used to retrieve specific or complete data from one or more database tables for analysis, reporting, or processing purposes.

Below is a clear breakdown of what each part of the **SELECT script** is doing at a database and query-execution level.

1. Selecting all data from employees

```
SELECT *  
FROM employees;
```

What is happening:

- **SELECT *** tells MySQL to retrieve **all columns**
- **FROM employees** specifies the table

Result:

You get the full employee table including:

- employee_id
- first_name
- last_name
- gender
- email
- department_id, etc.

Purpose:

Used for:

- Quick inspection of data
- Debugging
- Understanding table structure

2. Selecting a single column

```
SELECT first_name  
FROM employees;
```

**What is happening:**

- Only the first_name column is retrieved
- All other columns are ignored

Result:

A single-column output list of employee first names.

Purpose:

Used when you only need specific attributes to reduce data output.

3. Selecting multiple columns

```
SELECT first_name, last_name  
FROM employees;
```

What is happening:

- MySQL retrieves only first_name and last_name
- Data is returned row-by-row for each employee

Purpose:

- More efficient than **SELECT ***
- Common in reporting and dashboards

4. Selecting multiple columns (expanded version)

```
SELECT first_name, last_name, gender, hire_date  
FROM employees;
```

What is happening:

- A wider subset of employee attributes is retrieved
- Still only from one table

Purpose:

- Used for structured reporting
- HR-style summaries



5. Formatted SELECT (clean layout)

```
SELECT
    first_name,
    last_name,
    gender,
    hire_date
FROM employees;
```

Purpose:

- Improves readability
- Used in professional SQL scripts and GitHub projects

SQL ignores line breaks; this is purely for humans.

6. Calculated column (salary bonus example)

```
SELECT
    role_title,
    base_salary,
    base_salary + 1000 AS salary_bonus_example
FROM job_roles;
```

What is happening:

- base_salary is retrieved from the table
- A **calculated column** is created:
 - _salary + 1000
- AS salary_bonus_example renames the output column

Purpose:

- Shows derived values without changing the database
- Used in financial analysis and projections

7. Percentage increase calculation

```
SELECT
    role_title,
    base_salary,
    (base_salary * 1.10) AS increased_salary_10_percent
FROM job_roles;
```

**What is happening:**

- Each salary is multiplied by 1.10
- This simulates a **10% salary increase**
- Result is not stored, only displayed

Purpose:

- Business forecasting
- Salary planning
- Scenario analysis

8. Selecting values from a column (before DISTINCT)

```
SELECT department_id  
FROM employees;
```

What is happening:

- Returns all department IDs
- Includes duplicates

Example output:

```
2  
1  
3  
2  
1
```

Purpose:

- Raw data inspection
- Shows actual stored values

9. DISTINCT (removing duplicates)

```
SELECT DISTINCT department_id  
FROM employees;
```

What is happening:

- MySQL scans all values in department_id
- Removes repeated entries
- Returns only unique values

**Example output:**

```
1
2
3
```

Purpose:

- Data summarization
- Identifying unique categories

10. DISTINCT on department names

```
SELECT DISTINCT department_name
FROM departments;
```

What is happening:

- Retrieves all department names
- Ensures no duplicates appear

Purpose:

- Clean listing of categories
- Useful in reporting and dropdown menus

11. JOIN between employees and departments

```
SELECT
    e.first_name,
    e.last_name,
    d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

What is happening step-by-step:

- a. MySQL reads employees table (alias e)
- b. It reads departments table (alias d)
- c. It matches rows where:

```
employees.department_id = departments.department_id
```



- d. It combines matching rows into one result set

Result:

Each employee is shown with their department name.

Purpose:

- Relational data linking
- Core SQL concept (JOIN)

12. Multi-table JOIN (analytics query)

```
SELECT
    e.first_name,
    e.last_name,
    jr.role_title,
    jr.base_salary
FROM employees e
JOIN employee_positions ep
    ON e.employee_id = ep.employee_id
JOIN job_roles jr
    ON ep.role_id = jr.role_id;
```

What is happening step-by-step:**Step 1:**

- employees (e) is loaded

Step 2:

- employee_positions (ep) is joined using:

```
e.employee_id = ep.employee_id
```

Step 3:

- job_roles (jr) is joined using:

```
ep.role_id = jr.role_id;
```

Final result:



Each employee is linked to:

- their role
- their salary

Purpose:

- Real-world HR analytics
- Shows full employee profile across multiple tables

BIG PICTURE SUMMARY

The above scripts demonstrate 4 key SQL concepts:

1. Data Retrieval

- SELECT *
- SELECT columns

2. Data Transformation

- Calculations (+, *)
- Aliases (AS)

3. Data Filtering (Logical view)

- DISTINCT

4. Data Relationships

- JOIN operations

FINAL PROFESSIONAL VIEW

These scripts represent:

A full SQL data exploration workflow: from raw table inspection → column selection → transformation → deduplication → relational joins for analytics.